Data Engineering for AI and ML (DEAI)
02 Project Background

Dr. Arnab Phani
BIFOLD, TU Berlin
DEEM Lab

23.10.2026

Agenda

- **Logistics**
- **Recap. Machine Learning Engineering (MLE) Agents**
- **Project Background**
- **Example System: stratum**

Logistics

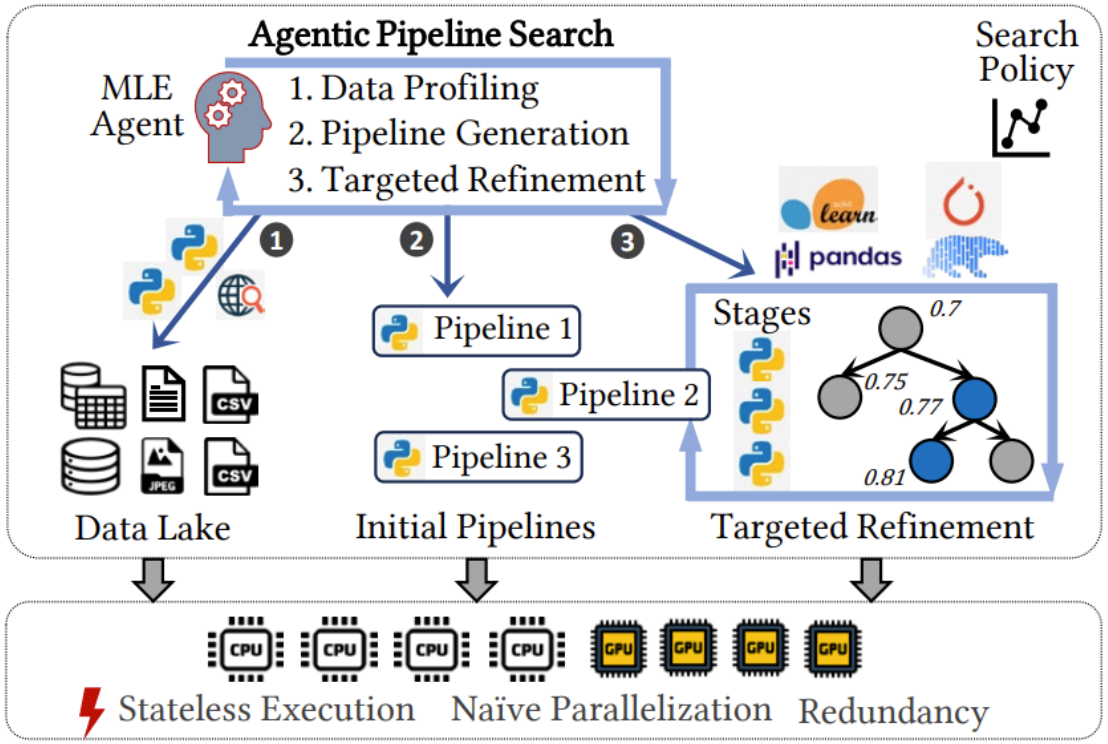
- **Finalize group selection by 30.10**
  - Group size: 3 – 5 students (4 is ideal)
  - Check ISIS for group selection
- **Pitch presentation: Nov 13**
  - 10 min presentation + 5 min QA
  - Location: **FR 710**
  - Problem insights, approach, prototypes, possible experiments, …
- **Optional consulting hours**
  - Wait for announcements

Timeline

Course admission: [16.10 – 23.10] ★
Project background: 23.10 ★
Finalize group selection: [23.10 – 30.10] ★
Independent planning: [23.10 – 30.10]
Brainstorming + Initial experiments: [30.10 – 13.11]
Prep pitch presentation: [06.11 – 13.11]
Pitch presentation: 13.11 ★
Initial prototype: [06.11 – 04.12]
Status report submission: 04.12 ★
Robustness and experiments: [04.12 – 15.01]
Final presentation: 15.01 ★
Improvements and fixes: [15.01 –12.02]
Project code and report: 12.02 ★

Project Background

Recap. Agentic Pipeline Search



Agentic Pipeline Search

Example Task Description for an MLE Agent

"Build the best end-to-end ML pipeline to predict customer churn from the datasets in this data lake."

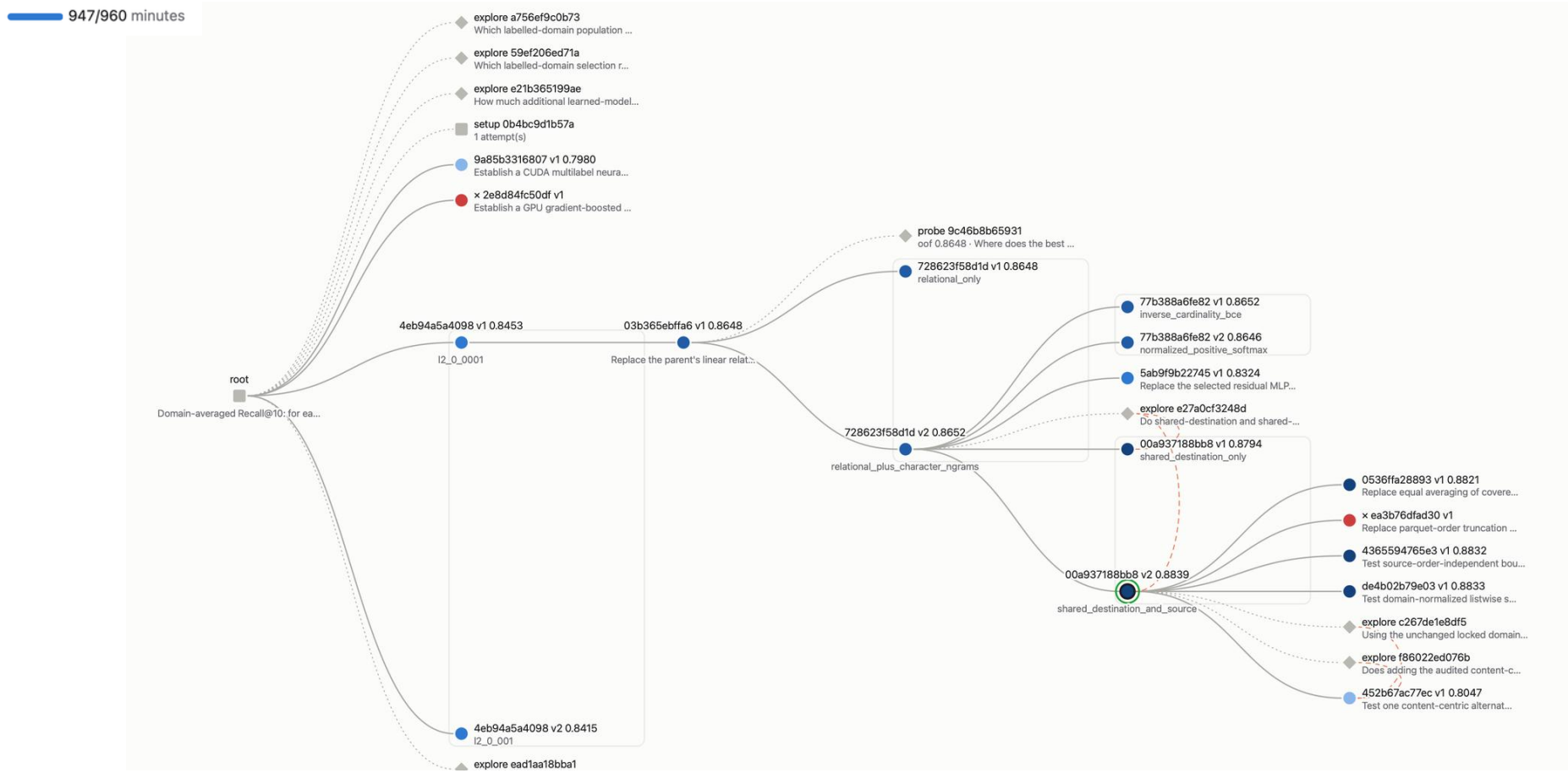
Simplified Code Generation & Execution Loop

```
for iteration in range(MAX_ITERATIONS):
    code = agent.generate_or_refine_code()
    exec(compile(code))
    agent.evaluate_and_submit()
```



- MLE agents explore many candidate solutions (in thousands)
- Data profiling, pipeline generation, targeted refinement; repeat
- Users often launch multiple agents with modified prompts

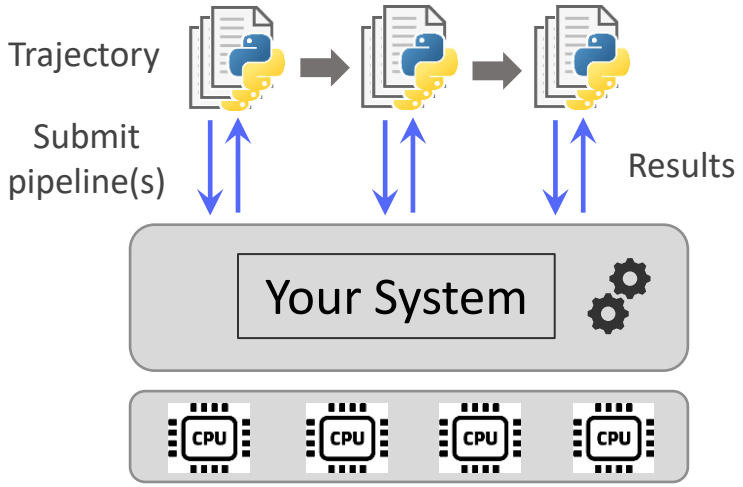
Recap. Trajectories



A **trajectory** is a sequence of actions and observations through which an MLE agent iteratively generates, evaluates, and refines ML pipelines to solve a given task.

Accelerating Agent-Generated ML Pipelines

Goal: Build a system that automatically optimizes agent-generated Python ML pipelines to reduce execution time while preserving predictive quality.



Open design: Each group chooses its own optimization strategies and system architecture

Possible Compile-Time Optimizations

- **Code and Compiler Optimizations**
  - *Library substitution:* Replace Pandas with Polars, NumPy with more efficient alternatives, etc.
  - *Code generation:* Translate Python operators or pipeline regions into optimized Rust code
  - *Rewriting:* Apply predicate pushdown, projection pruning, and redundant operator elimination
  - *Specialization:* Generate implementations optimized for known data types, shapes, or parameters
  - *Backend selection:* Choose between Python, native CPU implementations, and GPU execution

- **LLM-assisted Optimizations**
  - *Code translation:* Generate optimized Rust implementations from Python code
  - *Algorithm substitution:* Replace expensive algorithms with faster alternatives (correctness constraints)
  - *Kernel generation:* Synthesize specialized implementations for data prep, feature engineering
  - *Code repair:* Automatically diagnose compilation/runtime failures and regenerate implementations
  - *Optimization planning:* Use LLMs to identify optimization opportunities (systematic)
  - *Hybrid optimization:* Combine compiler-based transformations with LLM-generated implementations

Possible Runtime and Cross-Pipeline Optimizations

- **Runtime and Execution Optimizations**
  - *Intra-operator parallelism:* Parallelize expensive operations (e.g., Dask, Ray, n_jobs)
  - *Inter-operator parallelism:* Execute independent pipeline operations concurrently (e.g., n_jobs)
  - *Efficient data representation:* Reduce copying, serialization, unnecessary intermediates (e.g., in-place)
  - *Adaptive execution:* Select implementations based on data characteristics and runtime profiling

- **Cross-pipeline Optimizations**
  - *Intermediate result reuse:* Cache and reuse preprocessing or feature engineering results across pipelines
  - *Generated code reuse:* Reuse previously generated and validated Rust implementations.
  - *Common subexpression elimination:* Detect shared computations across different pipelines
  - *Incremental computation:* Update previously computed results when only part of a pipeline changes

Experimental Evaluation

- **Benchmark Workloads**
  - 10 pipeline trajectories (fixed sequence); 5 provided + 5 collected (at least 20 pipelines/trajectory)
  - Baseline: Original execution time and original predictive quality of a trajectory
  - Additional experiments encouraged: Vary dataset sizes, optimization settings, …

- **Evaluation Metrics**
  - Trajectory speedup: $S = T_{baseline} / T_{optimized}$
  - Report LLM inference time if used for optimization
  - Predictive quality: Best validation score achieved across the trajectory compared against the baseline (within a specified tolerance).

Example Trajectories

pipeline_1_1.py	trajectory 1
pipeline_1_2.py	trajectory 1
pipeline_1_3.py	trajectory 1
pipeline_2_1.py	trajectory 1
pipeline_2_2.py	trajectory 1
pipeline_2_3.py	trajectory 1
pipeline_3_1.py	trajectory 1
pipeline_3_2.py	trajectory 1
pipeline_3_3.py	trajectory 1
pipeline_4_1.py	trajectory 1
pipeline_4_2.py	trajectory 1
pipeline_4_3.py	trajectory 1
pipeline_4_4.py	trajectory 1
pipeline_4_5.py	trajectory 1

Naming: pipeline_<iteration ID>_<pipeline ID>



pipeline_1_* pipeline_2_* pipeline_3_*

Spaceship Titanic

Download the data from the Kaggle competition data page: sign in, accept the competition rules, and click **Download All**.

From the repository root, unzip the downloaded archive into the competition's `input/` folder:

```
mkdir -p spaceship_titanic_students/input
unzip ~/Downloads/spaceship-titanic.zip -d spaceship_titanic_students/input
```

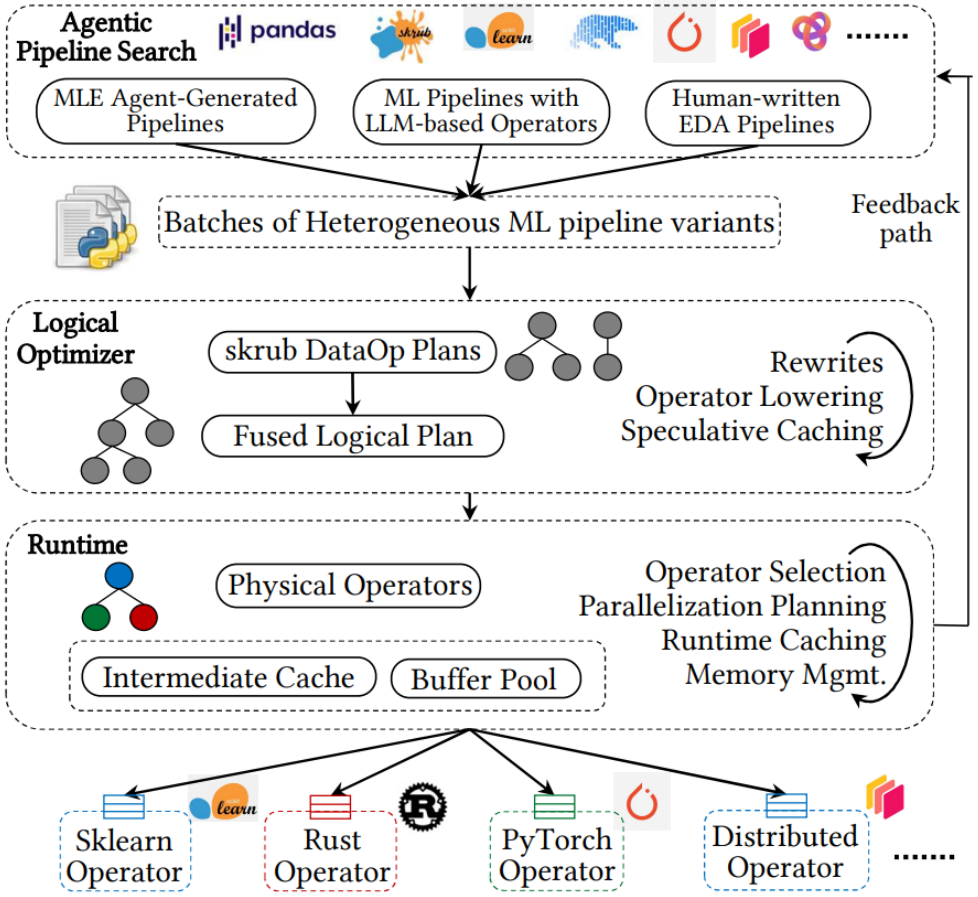
Adjust the archive path if needed. The CSV files should sit directly inside `spaceship_titanic_students/input/` (`train.csv` , `test.csv` , and `sample_submission.csv`). Run pipelines from `spaceship_titanic_students/` so they can find `input/` .

https://github.com/e-strauss/deai-pro

Example System: stratum

stratum: A System for Large-scale Agentic Pipeline Search

stratum



Pipelines are written in skrub DataOps format, allowing extensibility and composability

Lower DataOps IR to logical IR and apply logical rewrites

Lower logical IR to physical IR, apply operator selection and runtime optimizations

stratum: A System Infrastructure for Massive Agent-Centric ML Workloads

Arnab Phani
BIFOLD & TU Berlin
arnab.phani@tu-berlin.de

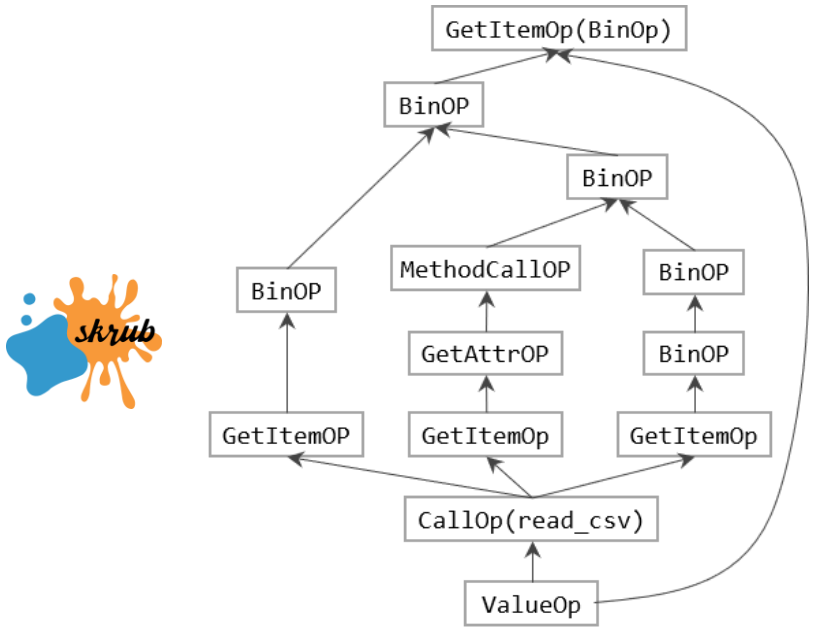
Elias Strauss
BIFOLD & TU Berlin
elias.strauss@tu-berlin.de

Sebastian Schelter
BIFOLD & TU Berlin
schelter@tu-berlin.de

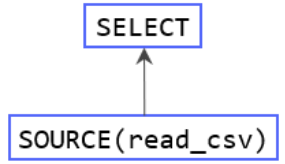
VLDB 2026 [Vision]

Logical Operators & Rewrites

stratum



Skrub DataOps → Logical Ops



SELECT

```
src = ...skb.apply_func(pd.read_csv)
pred = (src["age"] >= 25) & (src["age"] < 45) \
    & (src["name"].str.len() > 3) \
    & (src["income"] > 2000)
src_filtered = src[pred]
```

SQL-equivalent

```
SELECT *
FROM train
WHERE age >= 25
  AND age < 45
  AND LENGTH(name) > 3
  AND income > 2000;
```

- **Logical Operators and Rewrites**
  - Compile (raise) DataOps to Logical operators
  - Minimal set of operators (e.g., relational, transformer, predictor)
  - Apply **logical rewrites**

Physical Operator Selection

stratum

Physical Plan

```
=== Plan: physical_impl ===

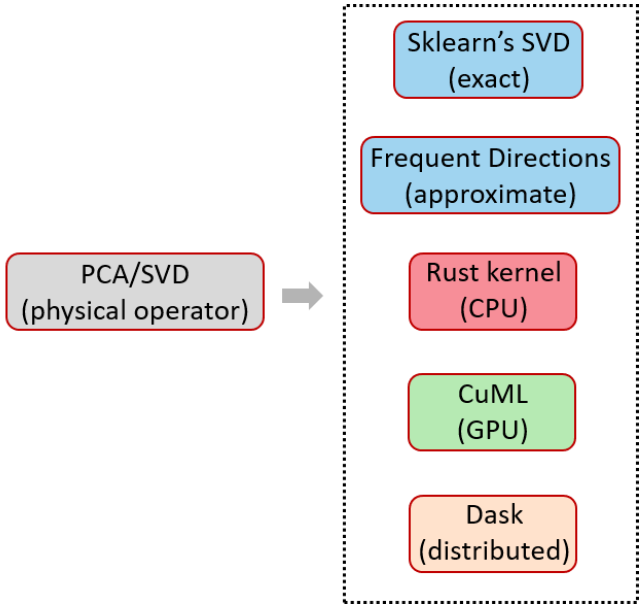
  [00] PolarsReadCSV(/tmp/tmphqrel2uy.csv) [df]   <- ()
  [01] PolarsSelectionOp(mask) [df]               <- ([00])
  [02] PolarsDropOp [df]                          <- ([01])
  [03] ImplOp(<X>)                                <- ([02])
  [04] PolarsColumnProjectionOp(cols['target'])   <- ([01])

CV Loop:
  [05] Split(Train/Test) [df]                     <- ([03], [04])

  Fit Phase:
    [06] SplitOutput(y) [df]                      <- ([05])
    [07] SplitOutput(X) [df]                      <- ([05])
    [08] PolarsAssignMapOp(assign: …) [df]        <- ([07])
    [09] PolarsColumnProjectionOp(cols[[…]]) [df] <- ([08])
    [10] StratumFusedTableVectorizer [df]         <- ([09])
    [11] Ridge [df]                               <- ([10], [06])

  Transform / Predict:
    [12] SplitOutput(y) [df]                      <- ([05])
    [13] SplitOutput(X) [df]                      <- ([05])
    [14] PolarsAssignMapOp(assign: …) [df]        <- ([13])
    [15] PolarsColumnProjectionOp(cols[[…]] [df]  <- ([14])
    [16] StratumFusedTableVectorizer [df]         <- ([15])
    [17] Ridge [df]                               <- ([16], [12])

Total: 5 pre-split  |  1 split  |  6 fit  |  6 predict
```



- Lower each logical operator to one or more physical operators
- A register maintaining **multiple implementations for each operator**; extensibility
- Operator selection to minimize compute time

Rust Backend

stratum

- **Problem**
  - Python-based ML operators introduce overhead
  - Type conversions, allocations, redundant copies
  - Often no multithreading
  - Usability focused; support a large set of configurations
- **Rust Backend**
  - Specialized Rust kernels for common operators
  - Py03 bindings with **GIL release**
  - **Zero-copy** data access and **multi-threading**
  - Focus on commonly used configurations
- **Additional Optimization Opportunities**
  - Accurate cost estimation
  - Operator fusion
  - Reduced boundary crossings
  - Fine-grained reuse

Table 3: Top learners and transformers in explicit pipelines.

	Top Transformers	Top Learners
SKLEARN	StandardScaler CountVectorizer TfidfTransformer PolynomialFeatures	LogisticRegression MultinomialNB SVC LinearRegression RandomForestClassifier

* Data Science Through the Looking Glass: Analysis of Millions of GitHub Notebooks and ML.NET Pipelines. SIGMOD Record. Vol 51 (2)

Stratum: A System for Agentic ML Pipeline Search



The stratum team

Paper

Code

BIFOLD

TECHNISCHE UNIVERSITÄT BERLIN